

Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

Esercizio 1

Si supponga che l'output tipico del comando "ls -l " (con opzione `-time-style="+%Y-%m-%d"`) sia:

```
drwxrwx--- 10 utente1  staff   4096 2008-02-10 14:28 Docs
drwxrwx--- 10 utente2  users   4096 2009-02-10 13:28 texasData
-rw-----  2  utente1  staff   4096 2007-02-10 14:28 doc1.pdf
-rw-r--r--  1  utente2  users   3145 2006-01-16 01:48 rel.tetex
-rw-----  1  utente2  users    268 2007-04-26 10:37 song1.mp3
-rw-r--r--  4  utente3  staff   4096 2006-08-28 19:29 text1.tex
-rw-r--r--  1  utente3  staff    11 2007-05-02 12:10 text2.tex
```

Utilizzando opportuni comandi in concatenazione si eseguano le seguenti operazioni

- (a) Contare quanti files hanno estensione ".tex"
- (b) Elencare i nomi dei files regolari appartenenti all'utente "utente1" oppure al gruppo "staff"
- (c) Stampare la dimensione totale occupata dai files regolari appartenenti al gruppo staff
- (d) Elencare i nomi dei files modificati tra il 2006 e il 2008

Esercizio 2

Si scriva uno script bash che, data un directory in input, crei una cartella per ogni gruppo associato ad almeno un file nella directory e in ognuna di queste una sotto-cartella associata ad ogni utente proprietario di almeno un file associato a tale gruppo. All'interno di queste ultime lo script sposterà tutti i file appartenenti al gruppo e di proprietà dell'utente. Esempio di output rispetto all'esempio precedente:

```
--staff
  |-- utente1
  |   |-- Docs
  |   |-- doc1.pdf
  |-- utente3
  |   |-- text1.tex
  |   |-- text2.tex
-- users
  |-- utente2
  |   |-- rel.tetext
  |   |-- song1.mp3
  |   |-- texasData
```


Le funzioni BASH

```
nomefunzione () {  
    # definisco variabili locali  
    local var1; local var2; ...  
    # leggo argomenti passati alla funzione  
    var1=$1  
    var2=$2  
    ...  
    # corpo della funzione  
    ...  
}
```

VALORI DI RITORNO:

- Print
- Comandi
- Variabili globali

Esercizio 3

Utilizzando opportuni comandi in concatenazione si eseguano le seguenti operazioni:

- (a) Usando awk, data una lista di stringhe separate dal carattere ":", sostituire la prima e seconda stringa con il carattere speciale "-". valori di stringhe non possono essere vuote
- (b) Scrivere l'equivalente di questo comando `echo "first:second:third"|awk -F ':' '{print \$2}'` con sed.
- (c) Dato un file stringhe.txt, estrarre tutte le righe contenenti la stringa "LSO" e "lso" e la riga subito dopo. Stamparle a video.
- (d) Dato un file fiori.txt contenente gli ordini di acquisto per fiori nella forma: nome_persona nome_fiore quantità costo. Leggere i dati *fiore_acquistato*, *quantità* e *costo*, stamparne su un'unica riga per ogni utente (nome_persona) i nomi dei fiori acquistati.

Esercizio extra

Utilizzando opportuni comandi in concatenazione si eseguano le seguenti operazioni:

- (a) Rimuovere tutti gli spazi bianchi riga per riga da un file
- (b) Crea un archivio tar compresso di tutti i file di tipo .log in una data directory
- (c) Mostrare i 10 processi attivi che consumano più memoria
- (d) Dato un file, se una riga contiene la stringa "elabo" scriverla al contrario "obale"

Esercizio 4

Scrivere uno script BASH "visita.sh" che visita tutti i file della directory corrente. Per ogni file determina il suo tipo. Se il file è leggibile è un file txt doc o un sh o un c, visita.sh chiede all'utente se si vuole vedere il suo contenuto mostrando solo le ultime 3 righe, mentre se il file è una sottodirectory, visita.sh chiede se si vuole visitare anche la sottodirectory. Per ogni file incontrato, visita.sh chiede se lo si vuole rimuovere prima di passare a gestire il prossimo file nella directory corrente.

Esercizio 5

Si realizzi uno script di shell BASH che analizzi una directory contenente file di log. Ogni file di log rappresenta il registro delle attività di un server per un determinato giorno e ha il seguente formato:

- (a) ORARIO è un orario nel formato HH:MM:SS.
- (b) UTENTE è il nome dell'utente che esegue un'azione.
- (c) AZIONE descrive l'azione dell'utente (come LOGIN, LOGOUT o ERROR).

Lo script deve accettare due argomenti: il percorso della directory contenente i file di log; e uno specifico UTENTE di cui devono essere analizzate le attività. Per l'utente dato:

- (a) Conta quante volte l'utente ha effettuato l'accesso (LOGIN), l'uscita (LOGOUT), e ha incontrato errori (ERROR).
- (b) Identifica gli orari di attività dell'utente (basato su ORARIO).
- (c) Se l'utente non ha attività registrate, lo script deve visualizzare un messaggio che lo indichi e uscire

Soluzioni 1

- `ls -l *.tex 2>/dev/null | wc -l`
- `ls -l | grep '^-' | awk '$3 == "utente1" || $4 == "staff" {print $9}'`
- `ls -l | grep '^-' | awk '$4 == "staff" {total += $5} END {print total}'`
- `ls -l --time-style=+%Y 2>/dev/null | awk '$6 >= 2006 && $6 <= 2008 {print $7}'`

Soluzioni 2

```
# Verifica che sia stata fornita una directory come argomento
if [ -z "$1" ]; then
    echo "Uso: $0 <directory>"
    exit 1
fi

# Directory fornita come input
input_dir=$1

# Verifica che la directory esista
if [ ! -d "$input_dir" ]; then
    echo "La directory $input_dir non esiste!"
    exit 1
fi

# Passiamo alla directory fornita
cd "$input_dir" || exit

# Elenco dei file regolari (usa ls per visualizzare i dettagli dei file)
file_list=$(ls -l)
```

```
# Ciclo per ogni gruppo unico

for group in $(echo "$file_list" | awk '{print $4}' | sort | uniq); do

    # Creiamo una directory per ogni gruppo

    mkdir -p "$input_dir/$group"

    # Ciclo per ogni utente associato al gruppo corrente

    for user in $(echo "$file_list" | awk -v grp="$group" '$4 == grp {print $3}' | sort | uniq); do

        # Creiamo una directory per ogni utente all'interno della cartella del gruppo

        mkdir -p "$input_dir/$group/$user"

        # Ciclo per spostare i file associati al gruppo e all'utente

        echo "$file_list" | awk -v grp="$group" -v usr="$user" '$4 == grp && $3 == usr {print $9}' | while read -r file; do

            # Spostiamo i file nella cartella corretta

            mv "$file" "$input_dir/$group/$user/"

        done

    done

done
```

Soluzioni 2

```
# Cicliamo sui gruppi unici

for group in $(ls -l $file_list | awk '{print $4}' | sort | uniq); do

    # Creiamo una directory per ogni gruppo

    mkdir -p "$input_dir/$group"


    # Cicliamo sugli utenti associati a questo gruppo

    for user in $(ls -l $file_list | awk -v grp="$group" '$4 == grp {print $3}' | sort | uniq); do

        # Creiamo una sottodirectory per ogni utente all'interno della directory del gruppo

        mkdir -p "$input_dir/$group/$user"


        # Spostiamo i file appartenenti al gruppo e all'utente nella corrispondente cartella

        find . -type f -group "$group" -user "$user" -exec mv {} "$input_dir/$group/$user/" \;


    done

done
```

Soluzioni 3

- `awk -F: '{ if ($1 != "" && $2 != "") { $1 = "-"; $2 = "-"; } print $0;}' OFS=":"`
OPPURE `awk -F: '{$1="-"; $2="-"; print $0}' OFS=":"`
- `echo "first:second:third" | sed 's/[^:]*:\([^:]*\).*/\1/'`
- `sed -n '/[Ll][Ss][Oo]/ {p; n; p}' stringhe.txt` OPPURE `awk '/[Ll][Ss][Oo]/ {print; getline; print}' stringhe.txt`
- `awk '{ fiori[$1] = (fiori[$1] ? fiori[$1] "," : "") $2 } END { for (nome in fiori) print nome, fiori[nome] }' fiori.txt`

Soluzioni 4

```
function handle_file {
    local file="$1"
    echo "File: $file"

    # Determina se il file è leggibile
    if [[ -r "$file" ]]; then
        # Controlla il tipo di file
        case "$file" in
            *.txt|*.doc|*.sh|*.c)
                echo "Il file è leggibile e di tipo testo o script."
                # Chiede se mostrare il contenuto
                read -p "Vuoi vedere le ultime 3 righe? (s/n): " show_content
                if [[ "$show_content" == "s" ]]; then
                    echo "Ultime 3 righe di $file:"
                    tail -n 3 "$file"
                fi
                ;;
            *)
                echo "Il file è leggibile ma non di tipo testo o script."
                ;;
        esac
    fi
}
```

Soluzioni 4

```
# Chiede se rimuovere il file
read -p "Vuoi rimuovere $file? (s/n): " remove_file
if [[ "$remove_file" == "s" ]]; then
    rm "$file"
    echo "$file rimosso."
fi
}

# Funzione per gestire le directory
function handle_directory {
    local dir="$1"
    echo "Sottodirectory: $dir"

    # Chiede se visitare la sottodirectory
    read -p "Vuoi visitare la sottodirectory $dir? (s/n): " visit_subdir
    if [[ "$visit_subdir" == "s" ]]; then
        visit_files "$dir"
    fi
}
```


Soluzioni 4

```
# Funzione per visitare i file nella directory
function visit_files {
    local dir="$1"
    # Scorre i file nella directory specificata
    for item in "$dir"/*; do
        # Controlla se l'elemento esiste (gestisce eventuali directory vuote)
        if [[ -e "$item" ]]; then
            if [[ -d "$item" ]]; then
                handle_directory "$item"
            else
                handle_file "$item"
            fi
        fi
    done
}

# Avvio dello script nella directory corrente
visit_files "."
```

Soluzioni 4-2

```
# Visita tutti i file nella directory corrente

for item in *; do

    # Controlla se l'elemento esiste (gestisce eventuali directory vuote)

    if [[ -e "$item" ]]; then

        if [[ -d "$item" ]]; then

            # Gestisce le sottodirectory

            echo "Sottodirectory: $item"

            read -p "Vuoi visitare la sottodirectory $item? (s/n): " visit_subdir

            if [[ "$visit_subdir" == "s" ]]; then

                # Visita la sottodirectory

                for subitem in "$item"/*; do

                    if [[ -e "$subitem" ]]; then

                        if [[ -f "$subitem" ]]; then

                            # Gestisce i file

                            echo "File: $subitem"

                            if [[ -r "$subitem" ]]; then

                                case "$subitem" in

                                    *.txt|*.doc|*.sh|*.c)

                                        echo "Il file è leggibile e di tipo testo o script."

                                        read -p "Vuoi vedere le ultime 3 righe? (s/n): " show_content

                                        if [[ "$show_content" == "s" ]]; then

                                            echo "Ultime 3 righe di $subitem:"

                                            tail -n 3 "$subitem"

                                        fi

                                    *)

                                        :

                                esac

                            fi

                        fi

                    fi

                done

            fi

        fi

    fi

done
```

Soluzioni 4

```
;;
*)
    echo "Il file è leggibile ma non di tipo testo o script."
;;
esac

fi

# Chiede se rimuovere il file
read -p "Vuoi rimuovere $subitem? (s/n): " remove_file
if [[ "$remove_file" == "s" ]]; then
    rm "$subitem"
    echo "$subitem rimosso."
fi

fi

fi

done

fi

else
```


Soluzioni 4

```
        echo "$subitem rimosso."
    fi
fi
fi
done
fi
else
    # Gestisce i file nella directory corrente
    echo "File: $item"
    if [[ -r "$item" ]]; then
        case "$item" in
            *.txt|*.doc|*.sh|*.c)
                echo "Il file è leggibile e di tipo testo o script."
                read -p "Vuoi vedere le ultime 3 righe? (s/n): " show_content
                if [[ "$show_content" == "s" ]]; then
                    echo "Ultime 3 righe di $item:"
                    tail -n 3 "$item"
                fi
            fi
        fi
    fi
fi
```

Soluzioni 4

```
;;
*)
    echo "Il file è leggibile ma non di tipo testo o script."
;;
esac

fi

# Chiede se rimuovere il file
read -p "Vuoi rimuovere $item? (s/n): " remove_file
if [[ "$remove_file" == "s" ]]; then
    rm "$item"
    echo "$item rimosso."
fi

fi

fi

done
```




Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

